STREAMS

Streams in Java

- Definition: A stream is a ordered sequence of data
- Unidirectional movement of data
- Sequential access to the data
- A stream represents a uniform, easy to use, object oriented interface between the program and the input/output devices
- Pipelining of streams
- Two types of streams:
  - Byte Streams
    - Input Stream/Output Stream
  - Character Streams
    - Reader classes/Writer classes

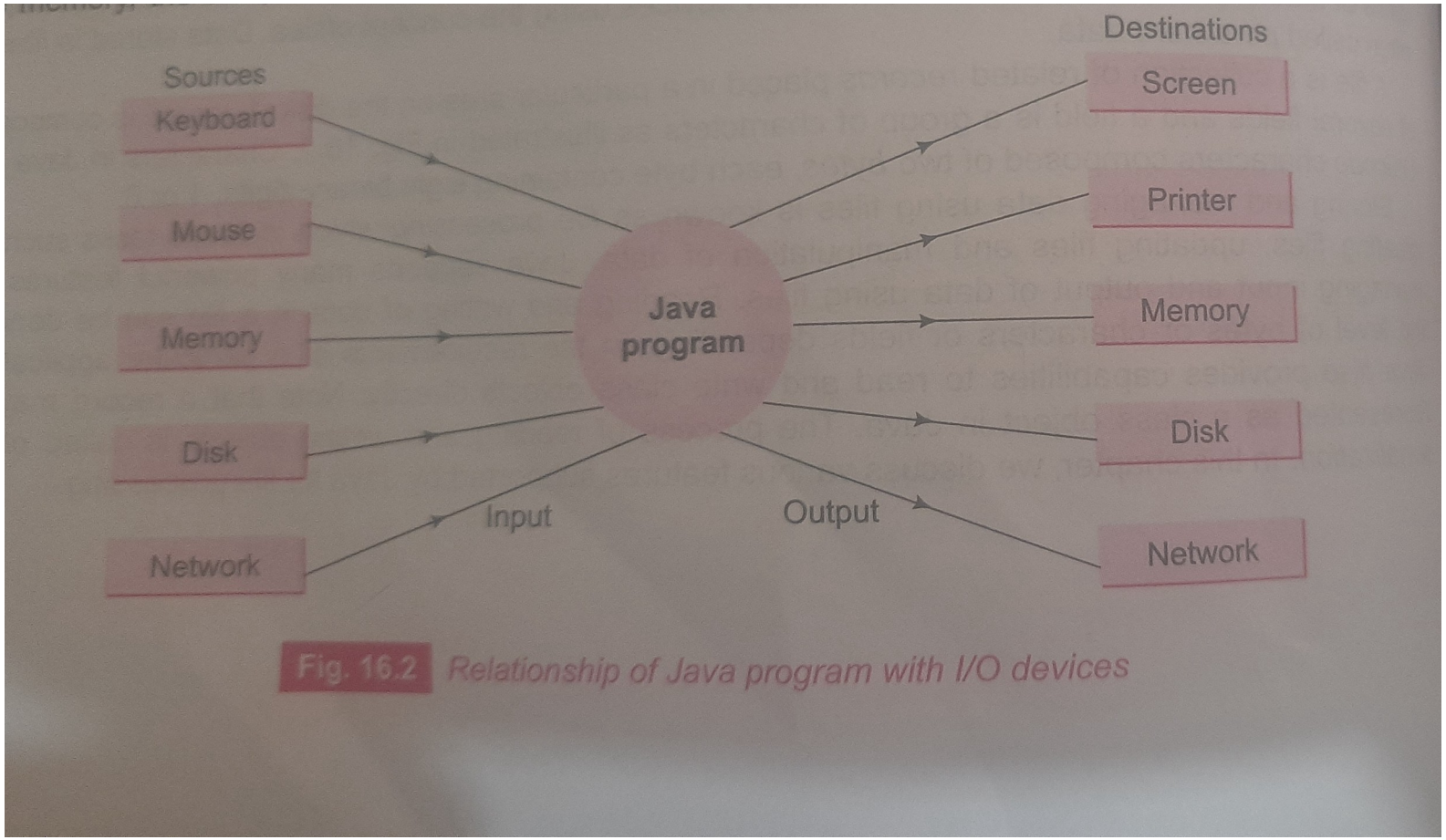
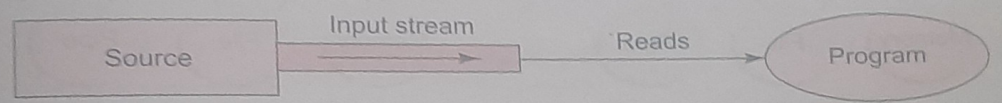


Fig. 16.2 *Relationship of Java program with I/O devices*

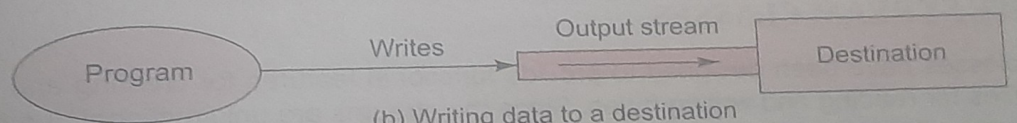
Conceptual view of a stream

The concept of sending data from one stream to another (like one pipe feeding into another pipe) has made streams in Java a powerful tool for file processing. We can build a complex file processing sequence using a series of simple stream operations. This feature can be used to filter data along the pipeline of streams so that we obtain data in a desired format. For example, we can use one stream to get raw data in binary format and then use another stream in series to convert it to integers.

Java streams are classified into two basic types, namely, *input stream* and *output stream*. An input stream extracts (i.e., *reads*) data from the source (file) and sends it to the program. Similarly, an output stream takes data from the program and sends (i.e., *writes*) it to the destination (file). Figure 16.4 illustrates the use of input and output streams. The program connects and opens an input stream on the data source and then reads the data serially. Similarly, the program connects and opens an output



(a) Reading data into a program



(b) Writing data to a destination

Fig. 16.4 *Using input and output streams*

Byte streams

- Used for reading and writing binary data
- Key classes:
  - InputStream (abstract class)
    - FileInputStream
    - ByteArrayInputStream
  - OutputStream (abstract class)
    - FileOutputStream
    - ByteArrayOutputStream

Character streams

- Used for reading and writing character data
- Key classes:
  - Reader (abstract class)
    - FileReader
    - BufferedReader
  - Writer (abstract class)
    - FileWriter
    - BufferedWriter

Reading and Writing to console

- System.in for input
- System.out for output

```
import java.util.Scanner;
public class ConsoleIO {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        System.out.println("Hello, " + name + "!");
        scanner.close();
    }
}
```

File class

- Java.io package includes File class for creating files and directories
  - Methods for creating a file
  - Opening a file
  - Closing a file
  - Deleting a file
  - Renaming
  - Getting the size etc

Input/output Exceptions

- On creating files and performing I/O operations on them I/O related exceptions are thrown
- EOFException
- FileNotFoundException
- InterruptedException
- IOException

Creation of Files

- Suitable name for the file (input.txt)
- Data type to be stored-characters, bytes, primitive type
- Purpose(reading,writing, updating)
- Method of creating the file

Inilitializing File Stream objects

- FileInputStream fis;

```
try
{
fis=new FileInputStream(“test.dat”)
}
catch(IOException e)
{
}
```

```
File infile;
infile=new infile(“test.dat”)

FileInputStream fis;
try
{
fis=new FileInputStream(infile)
}
catch(IOException e)
{
}
```

Reading and writing files

```
import java.io.*;
public class ReadFile {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader(”output.txt");
        BufferedReader br = new BufferedReader(fr);
        String line;
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }
        br.close();
    }
}
```

```
import java.io.*;
public class WriteFile {
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("output.txt");
        BufferedWriter bw = new BufferedWriter(fw);
        bw.write("Hello, Java I/O!");
        bw.close();
    }
}
```

File reader

- FileReader is a class in Java used for reading character files.
- It reads data one character at a time, making it relatively slow for large files.
- It does not have built-in buffering, meaning each read operation directly interacts with the file, leading to more I/O operations.

Buffered Reader

- BufferedReader is a wrapper around FileReader that adds buffering capabilities.
- It reads large chunks of data at once, reducing the number of I/O operations and improving efficiency.
- Provides methods like readLine() for reading an entire line at a time, making it more convenient for text processing.

Reading and writing bytes

- FileReader and FileWriter classes to read and write 16 bit characters
- FileInputStream and FileOutputStream reads 8 bit bytes

```
public class FileCopyBytes {
public static void main(String[] args) {
FileInputStream fin = null;
FileOutputStream fout = null;

try {
// Source file
fin = new FileInputStream("source.txt");
// Destination file
fout = new FileOutputStream("destination.txt");

int data;

// Read and write byte by byte
while ((data = fin.read()) != -1) {
fout.write(data);
}
}
```

```
 catch (IOException e) {
System.out.println("Error: " + e.getMessage());
```

- ```
  import java.io.FileInputStream;
  import java.io.IOException;

  public class ReadBytesExample {
  public static void main(String[] args) {
  try {
  FileInputStream fin = new FileInputStream("input.txt");

  int data;

  // Read byte by byte
  while ((data = fin.read()) != -1) {
  System.out.print((char) data); // convert byte to character
  }

  fin.close();
  } catch (IOException e) {
  System.out.println("Error: " + e.getMessage());
  }
  {
  {
  }
  ```
  ```

- ```
  import java.io.FileOutputStream;
  import java.io.IOException;

  public class WriteBytesExample {
  public static void main(String[] args) {
  try {
  FileOutputStream fout = new FileOutputStream("output.txt");

  String data = "Hello, this is a byte stream example!";
  byte[] bytes = data.getBytes(); // convert string to bytes

  fout.write(bytes); // write bytes to file

  fout.close();
  System.out.println("Data written successfully!");

  } catch (IOException e) {
  System.out.println("Error: " + e.getMessage());
  }
  }
  }
  ```
  ```

Handling primitive data types

- DataInputStream and DataOutputStream
- Filter classes which act as wrappers on existing input and output streams for handling primitive types
- FileInputStream fis=new FileInputStream (‘’text.dat”)
- DataInputStream dis=new DataInputStream(fis)
- Also for writing
- FileOutputStream fos=new FileOutputStream (‘’text.dat”)
- DataOutputStream dis=new DataOutputStream(fos)

```
import java.io.*;

public class DataStreamDemo {
    public static void main(String[] args) {
        try {
            // Writing primitive data types
            FileOutputStream fos = new FileOutputStream("data.txt");
            DataOutputStream dos = new DataOutputStream(fos);

            dos.writeInt(100);
            dos.writeDouble(45.67);
            dos.writeChar('A');
            dos.writeBoolean(true);

            dos.close();
            fos.close();

}
}
```

```
// Reading primitive data types
        FileInputStream fis = new FileInputStream("data.txt");
        DataInputStream dis = new DataInputStream(fis);

        int i = dis.readInt();
        double d = dis.readDouble();
        char c = dis.readChar();
        boolean b = dis.readBoolean();

        dis.close();
        fis.close();

        // Displaying values
        System.out.println("Integer: " + i);
        System.out.println("Double: " + d);
        System.out.println("Character: " + c);
        System.out.println("Boolean: " + b);

    } catch (IOException e) {
        System.out.println("Error: " + e);
    }
```

What is Object Serialization?

What is Object Serialization?

- **Object serialization** is the process of converting a Java object into a **byte stream** so that it can be:
  - Stored in a file
  - Sent over a network
  - Saved for later use

Deserialization = converting byte stream back into an object

- Saving a **student record** into a file
- Later reading it back exactly as it was
- class Student implements Serializable {

Writing object to the file

- ObjectOutputStream oos =

  new ObjectOutputStream(new FileOutputStream("file.dat"));

- oos.writeObject(obj);
- Object → converted into **bytes** → stored in file

Reading Object ← File (Deserialization)

- ObjectInputStream ois =

  new ObjectInputStream(new FileInputStream("file.dat"));

- Student s = (Student) ois.readObject();

- import java.io.*;

  // Step 1: Create a class that implements Serializable
  class Student implements Serializable {
  int id;
  String name;
  double marks;

  Student(int id, String name, double marks) {
  this.id = id;
  this.name = name;
  this.marks = marks;
  }

  void display() {
  System.out.println(id + " " + name + " " + marks);
  }
  }

- // Step 2: Main class
  public class ObjectFileDemo {
  public static void main(String[] args) {
  try {
  // Writing objects to file
  ObjectOutputStream oos =
  new ObjectOutputStream(new FileOutputStream("student.dat"));

  Student s1 = new Student(101, "Anu", 85.5);
  Student s2 = new Student(102, "Ravi", 78.0);

  oos.writeObject(s1);
  oos.writeObject(s2);

  oos.close();
  System.out.println("Objects written to file");
}

```
catch (Exception e) {
System.out.println(e);
}
}
}
```

- // Reading objects from file
  ObjectInputStream ois =
  new ObjectInputStream(new FileInputStream("student.dat"));

  System.out.println("Reading objects from file:");

  Student s;

  try {
  while (true) {
  s = (Student) ois.readObject();
  s.display();
  }
  } catch (EOFException e) {
  // End of file reached
  }

  ois.close();

Random Access File

- A **Random Access File** (RAF) in Java allows you to **read and write data at any position in a file**, unlike sequential files where you must read from the beginning.
- It is handled using the class:

RandomAccessFile (from java.io package)

- A file can be created and opened for random access by giving mode string as a parameter to the constructor when we open the file
- ‘r’ for reading only,’rw’ for reading and writing
- Supports a file pointer that can be moved to arbitrary position in the file prior to reading and writing
- seek () method
- file.seek(long position)
- position = number of **bytes from the beginning of the file**
- **seek(0)=beginning of the file**
- **seek(file.length())=end of the file**

```
import java.io.*;

class SeekDemo {
    public static void main(String args[]) throws IOException {

        RandomAccessFile file = new RandomAccessFile("data.dat", "rw");

        file.writeInt(10);  // 4 bytes
        file.writeInt(20);  // 4 bytes
        file.writeInt(30);  // 4 bytes

        // Move pointer to second integer
        file.seek(4);

        int value = file.readInt();
        System.out.println(value);
```